Built-in tools and structured output fundamentals

Connecting large language models to the real world

Large language models like Gemini are incredibly powerful. They can write stories, summarize articles, and answer complex questions. But they have a fundamental limitation: their knowledge is frozen in time. It's limited to the data they were trained on, like an encyclopedia that's never updated. It doesn't know about the morning's top news headline, the current price of a product on a shopping website, or the details of a technical document published yesterday. Its knowledge is frozen at the time of its last training.

To build helpful and intelligent applications, developers need a way to connect the model to the internet. This is where tools come in. Tools are functions that the model can choose to call when it needs information it doesn't have. Think of the model as a very smart, but offline, person. This information often comes back as a long, unstructured block of text—easy for a person to read but very difficult for a computer program to understand and use.

For most applications, performance, reliability, and cost are critical factors. Relying on a model to parse messy, unstructured text is slow, unpredictable, and expensive. This is the challenge that structured output solves. By combining tools to gather real-time information with a system for formatting that information into a clean, machine-readable schema, you can build applications that are fast, cost-effective, and dependable. This combination is a powerful pattern in agentic AI and is the key to moving beyond simple chatbots and into sophisticated, data-driven systems that can reason about the world and take meaningful actions. In this lesson, we'll explore how to use built-in tools like URL Context and Google Search, but the concepts you learn here will apply to any tools you create yourself.

Giving your AI access to the live web

To overcome the limitation of static training data, the Gemini API includes built-in tools that act as a bridge to external information. These tools allow the AI model to access and process information from the internet in real time. Instead of relying only on what it learned during training, the model can decide when it needs to use a tool like URL Context and Google Search. Each one is a different practical skill, with unique strengths and weaknesses.

While this module focuses on URL context and Google Search, it's important to note that the Gemini API supports a broader ecosystem of tools. These advanced capabilities include connecting to enterprise data sources like Google Maps, or providing the model with a secure environment to execute code for complex problem-solving. These powerful tools will be explored in more depth during a dedicated Gemini 3 Accelerate day, but they are not part of the core, introductory module.

The URL context tool

The URL context tool gives Gemini the ability to "read" the content of a specific webpage. When you provide a web address (URL) in your prompt and enable this tool, the model can fetch the text from that page and use it as context for its answer. This is incredibly useful when you need the AI to work with information from a known source.

For example, imagine you are building an application for a financial analyst who needs to quickly understand a company's Q3 quarterly earnings report. Without the URL context tool, the analyst would have to copy and paste the entire report into the prompt. With the tool, the developer can simply provide the URL to the report. The AI can then read the document directly and answer specific questions, like "What were the key findings concerning specific financial metrics, or answering questions about the company's performance.

Similarly, a customer support team can build an internal tool that answers questions about their company's complex, frequently updated product documentation. By using the URL context tool, Gemini can look at a broker that always has the most current information. When an employee asks, "What are the dispute configurations on the Lily Pro?" the tool can fetch the relevant help page and provide a precise, up-to-date answer, saving employees time and effort. Here's a diagram to show how the URL context tool works:

However, the URL context tool has important limitations.

- **Access restrictions:** It cannot access any Login, Paywalled, or Geo-restricted content (for example, a user in Europe cannot access a US-only news site).
- **Protected content:** It cannot bypass paywalls or login screens where valuable content is often located.
- **Dynamic content:** Some modern websites rely heavily on JavaScript to load their content. While the tool can process some of this, it may not work perfectly in every case.

Because of these limitations, developers must design their applications with fallbacks in place and should never rely on this tool for information that is absolutely critical and cannot be accessed by a person.

URL context process:

The Google search tool

While URL context is perfect for known sources, what if you need information about a current event or a topic you haven't researched yet? This is the job of the Google Search tool. It gives the model access to the same search engine that powers Google. It's the ideal tool when you don't have a specific URL to start with and need the model to investigate a topic. When the model needs information it doesn't have, it can perform a search, review the results, and use that information to form a more accurate and timely response. This is also known as "grounding" the model's answer in real-world facts and events, a form of fact-checking. The model's response is "grounded" in the information retrieved from the search.

For instance, a travel planning application could use Google Search to answer a user's question like, "What are some upcoming festivals in Kenya this month?" The model's training data would be out of date, but with Google Search, it can find current event listings and provide a list of festivals, complete with dates and locations. This is also a great tool for answering questions about recent news, sports scores, or the latest developments in a rapidly changing field. Here's a diagram of the model can use Google Search to find reliable news sources and respond to the current legislative status.

The main benefit of Google Search "grounding" is access to a nearly limitless and constantly updated source of information. However, this power comes with trade-offs.

- **Latency:** Using Google Search can increase the time it takes for the API to return a response, as the model has to perform the search and process the results.
- **Cost:** API calls cost. Pay-in-kinda.
- **Variability:** The quality of the output depends on the quality of the web's results. A search can sometimes return irrelevant or low-quality information, which could negatively affect the model's final response.

Developers must weigh the need for real-time information against these potential drawbacks.

Creating structured output with JSON Schema

Getting information from the web is the first challenge. Getting that data back in a way a machine can use is the next. By default, a model's response is just a string of text. If you ask it to summarize the financial analysis example, after reading the earnings report, it's in a readable but unstructured format.

"The company reported strong quarterly results, with revenues of $1.2 billion, which is a increase of 15% from last year. Net income was $250 million, and their earnings per share (EPS) was $1.25. The outlook for next quarter is positive."

This is perfect for a human, but frustrating for a machine. To use this information in software, it's just a string of text. Any software would, technically, be required to distribute it—it was the net income in a database, it would need complex code to "parse" this sentence. This process is fragile and can easily break if the model phrases its response differently next time.

This is where structured output becomes incredibly valuable. Structured output forces the model to provide its answer in a specific, machine-readable format, like the JSON (JavaScript Object Notation). The same financial data, structured in JSON, would look like this:

```
{
  "company_name": "Example Corp",
  "quarter": "Q3-2025",
  "revenue_growth_percent": 15,
  "net_income_usd": 250000000,
  "earnings_per_share": 1.25,
  "outlook": "positive"
}
```

This format is predictable and easy for any programming language to understand. To get the model to use this exact structure, you provide a JSON Schema in your request. The schema is a blueprint that tells the model the exact keys (like `revenue` or `eps`) it must include and what kind of data (like a number or a string) each one should contain. Modern Gemini models can also use Pydantic models or Zod schemas, which provide full autocomplete, type checking, and validation, making the process even smoother and more reliable.

JSON Schema transformation:

By defining a schema, you gain control and predictability. You can specify that revenue_growth_percent must be a number, that outlook must be a string, and that fields are required. This validation is a powerful shortcut for building robust applications. It reduces errors, simplifies your code, and builds a foundation of trust between the model and the rest of your system.

Key takeaways

- Large language models are limited by their static training data and require access to real-time information to be effective in modern applications.
- Built-in tools like URL Context and Google Search connect Gemini to the live internet, allowing it to work with up-to-date or specific external information.
- Raw information from these tools is often unstructured. Using a JSON Schema to enforce structured output is a critical technique for building reliable, machine-readable applications.
- The combination of real-time data retrieval and structured output is the fundamental building block for creating sophisticated autonomous AI agents that can analyze information and take meaningful actions.

Mark as completed

Like Dislike Report an issue